



OOPS - FAQ'S

110 INTERVIEW QUESTIONS & ANSWERS

Q1 What is OOP?

OOP (Object-Oriented Programming) is a programming style where we create classes and objects to organize code.

Q2 Why do we use OOP?

To make code:

Reusable

Easy to maintain

Easy to understand

Easy to extend

Q3 What is a class?

A class is a blueprint used to create objects.

EXAMPLE

```
class Student:  
    pass
```

Q4 What is an object?

An object is an instance of a class.

EXAMPLE

```
s = Student()
```

Q5 What is the difference between a class and an object?

Class → Blueprint

Object → Real thing created from the blueprint

EXAMPLE

Blueprint → House Plan

Object → Actual House

Q6 What is a constructor?

A constructor is a special method that automatically executes when an object is created.

```
def __init__(self):
```

Q7 Why is __init__() used?

To initialize object variables when the object is created.

Q8 What is self?

self refers to the current object.

Q9 Is self a keyword?

No.

It is just a naming convention.

You can write:

```
def __init__(x):
```

but using self is the Python standard.

Q10 Can we change the name of self?

Yes.

EXAMPLE

```
class Test:
    def __init__(x):
        print("Hello")
```

Works perfectly.

Q11 What is an instance variable?

A variable that belongs to an object.

EXAMPLE

```
self.name
```

Each object gets its own copy.

Q12 What is a class variable?

A variable shared by all objects.

EXAMPLE

```
class Student:  
    college = "ABC"
```

Q13 When should we use class variables?

When the value is common for all objects.

EXAMPLE

College Name

Company Name

Country

GST

Interest Rate

Q14 How do we modify a class variable?

```
Student.college = "XYZ"
```

or inside a class method using

```
cls.college
```

Q15 What happens if we modify a class variable using an object?

It creates a new instance variable instead of modifying the class variable.

Q16 What is an instance method?

A method that works with object data.

It always receives self.

Q17 What is a class method?

A method that works with class variables.

It uses:

```
@classmethod  
and receives cls.
```

Q18 What is a static method?

A method that belongs to the class but doesn't use instance variables or class variables.

EXAMPLE

Calculator methods like:

```
add()  
multiply()
```

Q19 Difference between instance, class, and static methods?

Method	First Parameter	Used For
Instance Method	self	Object data
Class Method	cls	Class data
Static Method	None	Utility methods

Q20 What is inheritance?

Inheritance allows one class to use the properties and methods of another class.

It promotes code reuse.

Q21 Why do we use inheritance?

To avoid writing the same code multiple times.

Q22 What is method overriding?

When a child class provides its own implementation of a parent class method.

Q23 What is super()?

super() is used to access the parent class methods or constructor.

Q24 What is encapsulation?

Encapsulation means binding data and methods together and controlling access to data.

Q25 What is a private variable?

A variable starting with double underscores.

```
| __salary
```

It cannot be accessed directly from outside the class.

Q26 What is polymorphism?

One interface, many implementations.

EXAMPLE

```
| dog.sound()  
| cat.sound()
```

Both use sound(), but produce different results.

Q27 What is abstraction?

Abstraction hides implementation details and shows only the essential features.

Q28 What are magic (dunder) methods?

Methods that start and end with double underscores.

EXAMPLE

```
| __init__()  
| __str__()  
| __len__()  
| __add__()
```

Python calls them automatically in specific situations.

Q29 What is __str__()?

It returns a readable string representation of an object.

Without it:

```
| <__main__.Student object at 0x...>
```

With it:

```
| Student(Name=Rahul)
```

Q30 What is __len__()?

It defines what should be returned when we call:

```
| len(object)
```

EXAMPLE

```
class Team:  
    def __len__(self):  
        return 11
```

Now,

```
print(len(team))
```

Output:

```
11
```

Q31 Can we create an object without writing a constructor?

Yes.

If you don't write `__init__()`, Python automatically provides a default constructor.

Q32 Can a class have more than one constructor?

No.

Python allows only one `__init__()` method in a class.

```
class Test:  
    def __init__(self):  
        pass  
    def __init__(self, x):  
        pass
```

The second constructor replaces the first one.

Q33 Can an object call a class method?

Yes.

Although class methods are meant for the class, objects can also call them.

```
obj.show()
```

Q34 Can a class call an instance method directly?

No.

Instance methods need an object.

```
Student.display() # Error (unless you pass an instance explicitly)
```

Q35 Can we create multiple objects from one class?

Yes.

A single class can create any number of objects.

Q36 Do all objects share instance variables?

No.

Each object has its own copy of instance variables.

Q37 Do all objects share class variables?

Yes.

There is only one copy of a class variable shared by all objects.

Q38 Where are instance variables stored?

Inside the object.

You can see them using:

```
| obj.__dict__
```

Q39 Where are class variables stored?

Inside the class.

You can see them using:

```
| ClassName.__dict__
```

Q40 What is the difference between obj.__dict__ and Class.__dict__?

```
| obj.__dict__ → Shows only instance variables.
```

```
| Class.__dict__ → Shows class variables and methods.
```

Q41 Can we delete an instance variable?

Yes.

```
| del obj.name
```

Q42 Can we delete a class variable?

Yes.

```
| del Student.college
```

Q43 What happens if we access a deleted variable?

Python raises an AttributeError.

Q44 Can we add variables after creating an object?

Yes.

```
class Student:
    pass
s = Student()
s.age = 20
print(s.age)
```

Python allows dynamic attribute creation.

Q45 Can we add methods after creating a class?

Yes.

Python classes are dynamic, and methods can be attached later.

(Though this is uncommon in real projects.)

Q46 What is id() in Python?

id() returns the unique memory identity of an object.

```
print(id(obj))
```

Q47 How do we know whether two variables refer to the same object?

Use the is operator.

```
a = b
print(a is b)
```

Returns:

```
True
```

Q48 What is the difference between == and is?

```
== compares values.
```

```
is compares object identity (memory reference).
```

EXAMPLE

```
a = [1, 2]
b = [1, 2]
print(a == b) # True
print(a is b) # False
```

Q49 Can one object be assigned to another variable?

Yes.

```
s1 = Student()
s2 = s1
```

Now both variables refer to the same object.

Q50 How do we count the number of objects created in a class?

Use a class variable.

```
class Student:
    count = 0
    def __init__(self):
        Student.count += 1
Student()
Student()
Student()
print(Student.count)
```

Output:

```
3
```

Q51 Why is Python called an Object-Oriented language?

Because almost everything in Python is an object, including:

int

float

string

list

tuple

function

class

EXAMPLE

```
print(type(10))
print(type("Python"))
print(type(len))
```

Q52 Is Python a purely Object-Oriented language?

No.

Python supports:

Object-Oriented Programming (OOP)

Procedural Programming

Functional Programming

It is a multi-paradigm language.

Q53 What is the default parent class of every Python class?

Every class implicitly inherits from:

```
| object
```

EXAMPLE

```
| class Student:  
|     pass  
| print(Student.__bases__)
```

Output

```
| (<class 'object'>,)
```

Q54 What is Method Resolution Order (MRO)?

MRO defines the order in which Python searches for methods during inheritance.

EXAMPLE

```
| class A:  
|     pass  
| class B(A):  
|     pass  
| class C(B):  
|     pass  
| print(C.mro())
```

Q55 What is the Diamond Problem?

It occurs in multiple inheritance when two parent classes inherit from the same base class.

Python solves this using MRO (C3 Linearization).

Q56 What is C3 Linearization?

C3 Linearization is the algorithm Python uses to determine the Method Resolution Order in multiple inheritance.

It ensures:

No duplicate searching

Predictable order

Consistent inheritance

Q57 What is the difference between `__new__()` and `__init__()`?

<code>new()</code>	<code>init()</code>
Creates object	Initializes object
Runs first	Runs second
Static method	Instance method

Q58 When should we override `__new__()`?

When we want to control object creation.

EXAMPLE

Singleton Design Pattern

Immutable objects

Custom object creation

Q59 What happens if `__new__()` doesn't return an object?

`__init__()` is never called.

Because the object was never created.

Q60 Can a constructor return a value?

No.

`__init__()` must return `None`.

Returning anything else raises:

`TypeError`

Q61 Why is Python called dynamically typed?

Because variable types are determined at runtime.

EXAMPLE

```
x = 10
x = "Python"
```

No datatype declaration is needed.

Q62 What is Duck Typing?

Python doesn't care about the object's type.

It only cares whether the object has the required method.

"If it walks like a duck and quacks like a duck, it's a duck."

Q63 What is Monkey Patching?

Changing or adding methods to a class at runtime.

EXAMPLE

```
Student.display = new_display
```

Q64 What are Magic Methods?

Methods surrounded by double underscores.

EXAMPLE

```
__init__()  
__str__()  
__repr__()  
__len__()  
__add__()  
__eq__()  
__iter__()
```

Python calls them automatically.

Q65 Difference between __str__() and __repr__()

str()	repr()
Human readable	Developer readable
Used by print()	Used in debugging
Friendly output	Official representation

Q66 What is operator overloading?

Giving new meaning to existing operators.

EXAMPLE

```
+  
-  
*  
==
```

using magic methods.

EXAMPLE

```
__add__()
```

Q67 Can we overload methods in Python?

No.

Python doesn't support traditional method overloading.

Instead we use:

Default arguments

*args

**kwargs

Q68 Why doesn't Python support method overloading?

Because Python allows:

Default values

Variable-length arguments

These features remove the need for traditional method overloading.

Q69 What is method overriding?

When a child class provides its own implementation of a parent class method.

Runtime polymorphism.

Q70 What is the difference between overloading and overriding?

Method Overloading	Method Overriding
Same method name with different parameters	Child redefines parent method
Not directly supported in Python	Fully supported
Compile-time concept (in many languages)	Runtime concept
Achieved using *args or default arguments	Achieved through inheritance

Q71 Can an abstract class have a constructor?

Yes.

Q72 Can an abstract class have normal methods?

Yes.

Q73 Can an abstract class have instance variables?

Yes.

Q74 Can we instantiate an abstract class?

No.

Q75 Can a child class skip implementing an abstract method?

No, unless the child class is also declared abstract.

Q76 What is composition in OOP?

One class contains another class as one of its attributes ("has-a" relationship).

Q77 What is aggregation?

A weak "has-a" relationship where objects can exist independently of each other.

Q78 What is the difference between composition and inheritance?

Inheritance: "is-a" relationship (e.g., Dog is an Animal)

Composition: "has-a" relationship (e.g., Car has an Engine)

Q79 Why is composition often preferred over inheritance?

It provides better flexibility, reduces tight coupling, and makes code easier to maintain.

Q80 What is the biggest advantage of OOP?

It helps build scalable, reusable, modular, and maintainable software by organizing related data and behavior into objects.

Q81 Why are classes themselves objects in Python?

In Python, everything is an object—including classes. A class is created by the type metaclass, so classes can be passed as arguments, returned from functions, and modified at runtime.

Q82 What is a metaclass?

A metaclass is a class that creates other classes. By default, Python uses the type metaclass.



EXAMPLE

```
class Student:  
    pass  
print(type(Student))
```

Output:

```
<class 'type'>
```

Q83 What happens internally when we create an object?

Python performs these steps:

Calls `__new__()` to create the object.

Allocates memory.

Calls `__init__()` to initialize the object.

Returns the object reference.

Q84 Can we stop object creation in Python?

Yes.

Override `__new__()` and return `None` instead of an object.

Q85 What is object identity in Python?

Object identity is the unique memory reference of an object.

Use:

```
id(obj)
```

Two different objects can have the same value but different identities.

Q86 Why is `is` faster than `==`?

```
is
```

 compares memory addresses.

```
==
```

 compares object values and may invoke `__eq__()`.

Memory comparison is generally faster.

Q87 What is the purpose of the `__slots__` attribute?

`__slots__` restricts dynamic attribute creation and reduces memory usage.

It is useful when creating millions of objects.

Q88 Can an object exist without any variable referring to it?

Yes.

If no variable references it, the object becomes eligible for garbage collection.

Q89 How does garbage collection relate to OOP?

Python automatically destroys objects that are no longer referenced, helping manage memory efficiently.

Q90 What is reference counting?

Python keeps a count of how many variables refer to an object.

When the reference count becomes zero, the object can be destroyed.

Q91 What is object introspection?

Introspection means examining an object at runtime.

Common functions:

```
type()
dir()
vars()
hasattr()
getattr()
setattr()
```

Q92 What is reflection in Python?

Reflection is the ability to inspect and modify objects dynamically at runtime.

EXAMPLE

```
getattr(obj, "name")
setattr(obj, "age", 25)
```

Q93 What is dynamic attribute creation?

Python allows adding attributes to an object after it has been created.

```
obj.city = "Hyderabad"
```

No need to define it inside the class.

Q94 What is dynamic method binding?

Methods are resolved at runtime based on the actual object's type.

This enables runtime polymorphism.

Q95 What is the difference between shallow copy and deep copy in OOP?

Shallow Copy → Copies only the outer object; nested objects are shared.

Deep Copy → Copies the object and all nested objects recursively.

Q96 What is object aliasing?

When two or more variables refer to the same object.

EXAMPLE

```
a = Student()  
b = a
```

Both variables point to the same object.

Q97 What is the difference between object equality and object identity?

Object Equality (==) → Compares values.

Object Identity (is) → Compares memory addresses.

Two objects may be equal but not identical.

Q98 What is late binding in Python?

Python resolves variable names at runtime, not when a function is defined.

This behavior is known as late binding.

Q99 Can private variables be accessed outside a class?

Not directly.

However, Python performs name mangling, so they can still be accessed (though it's discouraged).

EXAMPLE

```
obj._ClassName__salary
```

Q100 What is the biggest misconception about private variables in Python?

Many believe they are truly private.

Actually, Python uses name mangling, not strict access restrictions. Private variables are intended to discourage accidental access, not to enforce absolute security.

Q101 How does Python implement multiple inheritance internally?

Using the C3 Linearization algorithm to compute the Method Resolution Order (MRO).

Q102 What happens if two parent classes implement the same method?

Python follows the MRO to determine which method to call.

Q103 Why is Python considered highly dynamic compared to Java or C++?

Classes, objects, methods, and attributes can all be created, modified, or replaced at runtime.

Q104 Can methods be stored inside variables?

Yes. Methods are first-class objects and can be assigned to variables or passed as arguments.

Q105 What is the difference between mutable and immutable objects in OOP?

Mutable: Can be modified after creation (e.g., list, dict, set).

Immutable: Cannot be modified after creation (e.g., int, str, tuple).

These questions focus on Python's object model, runtime behavior, memory management, and advanced OOP features that are frequently discussed in senior-level interviews.

Q106 Why is Python called a "fully object-oriented" language?

Because almost everything in Python (integers, strings, lists, functions, classes, etc.) is treated as an object.

Q107 Is every class an object in Python?

Yes. Classes themselves are also objects, created by the metaclass type.

Q108 What is the default parent class of every Python class?

object

EXAMPLE

```
class Student:
    pass
print(Student.__bases__)
```

Output:

```
| (<class 'object'>,)
|
```

Q109 What is the difference between creating an object and initializing an object?

```
| __new__() creates the object.
| __init__() initializes the object with data.
|
```

Q110 Can we call a constructor manually?

Yes.

```
| class Student:
|     def __init__(self):
|         print("Constructor Called")
| s = Student()
| s.__init__()
|
```

The constructor message will be printed twice, though manually calling `__init__()` is rarely needed in practice.

These additional questions cover Python-specific OOP behavior that students often encounter in interviews but don't usually see in introductory OOP lessons.

Bonus Rapid-Fire Questions

Is Python an OOP language? → Yes

Can Python support multiple inheritance? → Yes

Can a class have multiple objects? → Yes

Can an object belong to multiple classes? → No

Can we create multiple objects from one class? → Yes

Is self a keyword? → No

Is cls a keyword? → No

Which method executes automatically when an object is created? → `__init__()`

Which method is called by `print(object)`? → `__str__()`

Which method is called by `len(object)`? → `__len__()`

Which decorator is used for class methods? → `@classmethod`

Which decorator is used for static methods? → `@staticmethod`

Which method accesses parent class methods? → `super()`

Which variable is shared by all objects? → Class variable

Which variable is unique to each object? → Instance variable

FOLLOW US

Scan the QR codes to connect with us



 **Instagram**
@vpro.skills



 **YouTube**
@vpro.skills

